Python Deep Dive: Round 1 (Interviewer A Prep)

Detailed answers to Q1–Q5, written so you can learn the mechanism and then say it out loud. The code samples were run (Python 3.9.6 + Pydantic v2) and the outputs shown are real.

Contents

Python Deep Dive: Round 1 (Interviewer A Prep)
- Contents
- How to use this document
- Q1. TypedDict vs dataclass vs Pydantic BaseModel
- Q2. Mutable default arguments (and the class-attribute twin)
- Q3. GIL, threads, processes and asyncio
- Q4. Generators and memory
- Q5. A @retry decorator, functools.wraps, and what must not be retried
- One-page recall sheet
- Self-test (cover the answers above)
- Next rounds (pending)

How to use this document

1. For each question, read **"The 30-second answer"** first. That is what you say out loud.
2. Read **"Why it works that way"** so you can survive the follow-up.
3. Read **"Where it bites in your work"**. This ties the concept to ResolveFlow or Company W so your answer sounds lived-in, not textbook.
4. Close the doc and re-answer from memory. Then check **"Follow-ups he may ask"**.

> **Honesty rule for the interview.** Only claim what you actually did. Where this document says "in ResolveFlow you would…", check the repo first. If you didn't do it, say "I'd design it this way" rather than "I built it".

Q1. TypedDict vs dataclass vs Pydantic BaseModel

The 30-second answer

All three describe "a shape of data", but they differ in **who enforces the shape and when**.

- `TypedDict` is a **plain `dict` at runtime**. The annotations are only for the type checker (mypy/pyright). Nothing is validated, and nothing stops you putting a string in an `int` field.

- `dataclass` generates `__init__`, `__repr__` and `__eq__` for a normal class. **Annotations are still not enforced at runtime.** It is a convenience for writing classes, not a validator.
- Pydantic `BaseModel` **parses and validates at runtime**. It coerces types, raises `ValidationError` with a precise error list, and can emit a JSON Schema.

My rule: **validate at the boundary, trust the inside.** LLM output is untrusted input, so it goes through Pydantic. Graph state that my own nodes produce is trusted and needs to be cheap, partial and serializable, so `TypedDict` is enough.

Proof (real output)

```
from typing import TypedDict
from dataclasses import dataclass, field
from pydantic import BaseModel, ValidationError

class S(TypedDict):
    n: int
s: S = {"n": "not an int"}          # no error at runtime
print(s)                             # {'n': 'not an int'}

@dataclass
class D:
    n: int
    tags: list = field(default_factory=list)
print(D(n="x"))                      # D(n='x', tags=[])  <- still no error

class M(BaseModel):
    n: int
print(M(n="42"))                     # n=42     <- coerced "42" -> 42
M(n="abc")                           # ValidationError: 1 validation error
```

Why it works that way

	`TypedDict`	`dataclass`	`BaseModel` (Pydantic v2)
Runtime type	plain `dict`	normal class instance	normal class instance
Runtime validation	**none**	**none** (only a manual `__post_init__`)	**yes**, on construction
Coercion	none	none	yes in default "lax" mode (`"42"` → `42`); `strict=True` turns it off
Attribute access	`s["n"]`	`d.n`	`m.n`
Partial/missing keys	`total=False`, `NotRequired[...]`	needs defaults	`Optional`/defaults, otherwise `ValidationError`
JSON Schema	no	no	`Model.model_json_schema()`
Parse from JSON string	no	no	`Model.model_validate_json(text)`
Overhead	zero (it is a dict)	tiny	highest (validation work on every build)
Immutability	no	`frozen=True`	`model_config = ConfigDict(frozen=True)`

Key mechanics worth being able to explain:

- **TypedDict is erased at runtime.** `isinstance(x, MyTypedDict)` raises `TypeError`. It exists purely so a static checker can verify key names and value types.
- **Dataclass mutable defaults are rejected.** `tags: list = []` raises `ValueError: mutable default <class 'list'> for field tags is not allowed: use default_factory` (I ran it). That check exists to prevent the shared-default bug in Q2.
- **Pydantic v2's validation core (`pydantic-core`) is written in Rust**, which is why it is fast enough to use on every request. It is still slower than a bare dict.
- **Lax vs strict mode.** In lax mode `"42"` becomes `42` and `"true"` becomes `True`. For security-sensitive input (permissions, IDs, money) you may want `strict=True` so a surprising string doesn't silently become something else.

Why Pydantic for LLM output but TypedDict for graph state

LLM structured output → Pydantic, because it is a trust boundary.

1. The model can return the wrong type, a missing field, an extra field, or a value outside the allowed set. `ValidationError` tells you exactly which field failed.
2. `model_json_schema()` gives you the schema you hand to the model (function calling / tool definitions). One class is both the **contract you send** and the **validator you receive with**. They cannot drift apart.
3. The error is machine-readable, so you can feed it back to the model for a repair attempt ("field `severity` must be one of low/medium/high").
4. `Literal["low","medium","high"]` or an `Enum` makes the allowed values explicit. That maps to your resume's "operation allowlists" and "schema validation".

LangGraph state → TypedDict, because it is internal and updated in pieces.

1. Nodes return **partial updates** (`{"diagnosis": ...}`), not a full object. A dict with optional keys fits that naturally.
2. LangGraph reads the **annotations** to decide how to merge updates. `Annotated[list, operator.add]` or `Annotated[list, add_messages]` attaches a **reducer** to a key. The annotation is data the framework inspects at graph-build time.
3. State is checkpointed and serialized on every step. A plain dict is the cheapest thing to serialize.
4. Your own nodes produced it. There is nothing untrusted to validate, and validating on every step is overhead.

```
import operator
from typing import Annotated, Optional, TypedDict

class TriageState(TypedDict):
    issue_text: str
    evidence: Annotated[list[str], operator.add]   # reducer: concatenate lists
    plan: Optional[dict]                            # overwritten on update (no reducer)
```

LangGraph also accepts a dataclass or a Pydantic model as the state schema. I'd check the docs for your installed version on exactly when validation runs between nodes before claiming more than "it validates the input it receives".

Where it bites in your work

- Your resume says "schema validation, output validation, operation allowlists". The honest mapping is: tool arguments and model-produced plans are parsed through a schema (Pydantic or JSON Schema) **before** any GitHub or Company W write. If you can't point at the file where that happens, describe it as the design.
- "Approval-gated execution": the approved plan should be a **frozen** Pydantic model, so nothing can mutate it between the human clicking Approve and the write layer executing it. That is a concrete, defensible use of `frozen=True`.

Follow-ups he may ask

- "*Pydantic validates on construction. What about assignment later?*" By default, no. Set `validate_assignment=True` to re-validate on `m.n = ...`.
- "*Why not Pydantic everywhere?*" Overhead on hot paths, and it adds a second source of truth if the data is only ever internal.
- "*`model_validate` vs `model_construct`?*" `model_construct` skips validation. Dangerous on untrusted data; it exists only for speed with data you already trust.
- "*`Optional[int]` vs default `None`?*" `Optional[int]` means "may be None", not "may be omitted". In Pydantic v2, required-but-nullable and optional-with-default are different.
- "*What does `from __future__ import annotations` do to Pydantic/LangGraph?*" It turns annotations into strings. Both libraries must resolve them back with `typing.get_type_hints`, which can fail for locally defined types. A classic source of confusing errors.

Q2. Mutable default arguments (and the class-attribute twin)

The 30-second answer

```
def add_item(item, bucket=[]):
    bucket.append(item)
    return bucket

print(add_item("a"))   # ['a']
print(add_item("b"))   # ['a', 'b']   <- surprise
```

Default values are evaluated **once, when the `def` statement runs**, not each time the function is called. That one list object is stored on the function and reused by every call that doesn't pass `bucket`. In a multi-tenant service this is not a quirk, it is a **cross-tenant data leak**: tenant B's request can see tenant A's data.

Why it works that way

`def` is an executable statement. When Python executes it, it evaluates the default expressions and stores the results in `function.__defaults__`. You can see it:

```
print(add_item.__defaults__)   # (['a', 'b'],)   <- the same list, growing
```

The function object owns that list. Calls that don't supply the argument all receive a reference to it. Mutating it (`append`) mutates the shared object. Rebinding the name (`bucket = bucket + [item]`) would **not** leak, because that creates a new list. The bug needs **mutation of a shared default**.

A trap if you test this yourself: `print(add_item("a"), add_item("b"))` prints `['a','b'] ['a','b']`, because both calls return the *same* list object and the print happens after both mutations. Two separate `print` calls give the clean `['a']` then `['a','b']`. I hit this exact thing while verifying this document.

The idiomatic fix

```
def add_item(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

`None` is immutable, so sharing it is harmless, and the real list is created **inside** the call, fresh every time.

If `None` is itself a legitimate value the caller might pass, use a private sentinel:

```
_MISSING = object()

def f(x, bucket=_MISSING):
    if bucket is _MISSING:
        bucket = []
    ...
```

The class-level twin

```
class TenantCache:
    cache = {}                  # ONE dict, shared by every instance and every tenant

    def put(self, k, v):
        self.cache[k] = v       # mutates the class attribute
```

`self.cache[k] = v` looks up `cache` on the instance, doesn't find it, finds the class attribute, and mutates it. Every instance writes to the same dict.

Fixes:

```
class TenantCache:
    def __init__(self):
        self.cache = {}         # per-instance

@dataclass
class TenantCache:
    cache: dict = field(default_factory=dict)   # dataclass REFUSES `= {}` and tells you to do this
```

Other places the same bug hides

Hiding place	Why it is the same bug	Safe pattern
Module-level `_cache = {}`	One per process, shared by all requests/tenants	Key every entry by tenant id, or keep request state in a `contextvars.ContextVar`

Hiding place	Why it is the same bug	Safe pattern
`@lru_cache` / `@functools.cache` on a function whose result depends on the caller's permissions	Cache key is only the arguments. If tenant/user isn't an argument, tenant B gets tenant A's cached answer	Make `tenant_id` / `user_id` an explicit argument so it becomes part of the key
Returning an internal list/dict from a method	Caller can mutate your internal state	Return a copy or an immutable view (`tuple` , `MappingProxyType`)
Shallow copy: `copy.copy(d)` / `dict(d)` / `list(l)`	Inner objects are still shared	`copy.deepcopy` when nested state matters
`[[]] * 3`	Three references to one inner list	`[[] for _ in range(3)]`

Where it bites in your work

- **Agent state.** If a node's default evidence list is a shared mutable, one run's findings appear in the next run's prompt. For an access-management agent that is the exact failure mode your "permission-aware execution" claim is about.
- **Per-user context.** For anything request-scoped (current user, tenant, trace id) the correct tool is `contextvars.ContextVar` . It is isolated per asyncio task and per thread, which a module global is not. This is also how you would propagate correlation attributes like `tenantId` and `traceId` without threading them through every function signature. (That's Karan's own vocabulary from his observability writing, so it lands well.)

Follow-ups he may ask

- "*Is it ever useful?*" Occasionally as a deliberate memo cache (`def f(x, _memo={})`), but it is obscure. `functools.lru_cache` states the intent better.
- "*Why does Python do it this way?*" `def` is executed once. Evaluating defaults per call would be hidden extra work and would make `def f(x=expensive())` surprising the other way. It is a consistent rule: defaults are evaluated at definition time.

Q3. GIL, threads, processes and asyncio

The 30-second answer

The **GIL (Global Interpreter Lock)** is a mutex in CPython that lets only **one thread execute Python bytecode at a time**. It exists to protect the interpreter's internal state, mainly reference counts, which are not thread-safe.

What I use:

- **`asyncio`** : many concurrent I/O operations (hundreds of LLM or API calls) when the libraries have async clients. One thread, cooperative switching at `await` .
- **`threading`** : I/O-bound work with blocking libraries (`requests` , `boto3` , some DB drivers). The GIL is **released while a thread waits on I/O**, so threads genuinely overlap.

- `multiprocessing` / `ProcessPoolExecutor`: CPU-bound pure-Python work (parsing huge files, chunking, scoring). Separate processes, each with its own GIL, so they use multiple cores.

For my workload (LLM calls, GitHub API, Tavily search) everything is I/O-bound, so **the GIL does not matter**. The time is spent waiting on the network, and waiting releases the GIL.

Why it works that way

What the GIL does and doesn't do

- It serialises execution of Python bytecode across threads in one process. CPU-bound threads therefore don't speed up; they can even slow down from lock contention.
- It is **released** during blocking I/O (socket reads, file reads, `time.sleep`) and inside many C extensions (NumPy, hashing, compression) while they run native code.
- Threads are **preemptive**: the interpreter forces a switch at a fixed interval (`sys.getswitchinterval()`, 5 ms by default), so a thread can be interrupted between *any* two bytecodes.

The GIL does not make your code thread-safe. It protects the interpreter, not your logic. `counter += 1` compiles to several bytecodes (load, add, store), and a switch can land in the middle. I ran this on Python 3.9.6:

```
c = 0
def inc():
    global c
    for _ in range(1_000_000):
        c += 1

# 4 threads each running inc()
print(c)    # 3227333   (expected 4000000)  <- lost updates
```

The exact number varies by run and Python version (newer versions changed where switches can happen and may hide it, which is the point: relying on that is a bug waiting to happen). Fix with `threading.Lock`, a `queue.Queue`, or by not sharing mutable state.

asyncio in one paragraph

An event loop runs on one thread. Each coroutine runs until it hits an `await` on something that isn't ready, then hands control back so another coroutine can run. There are no data races on plain variables between awaits (switches only happen at `await`), but **one blocking call freezes everything**. `time.sleep(5)` or a synchronous `requests.get()` inside `async def` stalls every other task for those seconds.

```
import asyncio, httpx

async def fetch_all(urls):
    async with httpx.AsyncClient(timeout=10) as client:
        sem = asyncio.Semaphore(10)                # cap concurrency (rate limits!)
        async def one(u):
            async with sem:
                r = await client.get(u)
                return r.status_code
        return await asyncio.gather(*(one(u) for u in urls))
```

For a blocking library you can't replace, push it off the loop: `await asyncio.to_thread(blocking_fn, arg)`.

Decision table

Workload	Best tool	Why
200 concurrent LLM/API calls	`asyncio` (+ semaphore)	Cheap tasks, waiting dominates, easy to cap concurrency
Blocking SDK (`boto3`, sync DB driver)	`ThreadPoolExecutor` or `asyncio.to_thread`	GIL released on I/O, no need for an async client
Parse/transform 50 GB of logs in pure Python	`ProcessPoolExecutor`	CPU-bound, needs real parallelism
Numeric work in NumPy / local embedding model	Threads are fine	The heavy part runs in native code with the GIL released
Shared mutable counter across workers	Lock, `Queue`, or per-worker partial results merged at the end	GIL doesn't protect compound operations

Costs of each

- **Threads**: shared memory (convenient, race-prone), about MBs of stack each, practical limit of hundreds to low thousands.
- **Processes**: isolated memory (safe, heavy), arguments and results must be **pickled** and copied between processes, startup is expensive on macOS/Windows (`spawn`).
- **asyncio**: tens of thousands of cheap tasks, but it spreads through the codebase (async can only be awaited from async), and every dependency in the hot path must be async-aware.

Where it bites in your work

- **LangGraph fan-out.** Parallel branches are I/O-bound LLM or retrieval calls, so concurrency (not parallelism) is what you need. Say that precisely.
- **Rate limits.** Unbounded `gather` over 500 items will trip provider limits (HTTP 429). A semaphore plus retry-with-jitter is the answer, not "more threads".
- **Local open-weight models (Ollama).** Inference runs in a separate server process. Your Python client is still just waiting on I/O.
- **Where the GIL genuinely hurts you**: chunking and cleaning a large corpus in pure Python before embedding. That is when you reach for `multiprocessing`.

The modern note (shows currency)

Python 3.13 introduced an **experimental free-threaded build** (PEP 703) with the GIL removed, and later releases are moving it toward supported status, but it is **not the default** and much of the C-extension ecosystem is still catching up. In production today you plan around the GIL. Say you know it exists and that you would not bet a production system on it yet. (Verify the exact status for the current release before quoting versions.)

Follow-ups he may ask

- "*Is `asyncio` parallel?*" No. It is concurrent, not parallel: one thread, one core.
- "*`gather` vs `TaskGroup`?*" `asyncio.TaskGroup` (3.11+) gives structured concurrency: if one task fails, siblings are cancelled and the errors are raised together. `gather` by default leaves the others running unless you handle it.
- "*What happens to a cancelled task?*" `CancelledError` is raised at the next `await`. Let it propagate (or re-raise after cleanup) and use `try/finally` so locks and connections are released.
- "*Why not threads for 10,000 connections?*" Memory per thread and context-switch cost. That is the C10K argument for event loops.
- "*Deadlock in threads vs stall in asyncio?*" Threads deadlock on lock ordering. Asyncio stalls from blocking calls or awaiting something that never completes. Always put timeouts (`asyncio.timeout()` / `wait_for`).

Q4. Generators and memory

The 30-second answer

A function containing `yield` doesn't run when you call it. It returns a **generator object**. Each `next()` resumes the function from where it last paused, runs to the next `yield`, hands back that value, and **freezes its local state** again. You produce values **lazily, one at a time**, with constant memory.

For 2 million log lines: a list would hold all of them in RAM before you start. A generator holds **one line at a time**, starts producing results immediately, and can stop early (stop at the first 10 failures without reading the rest).

After it is exhausted, iterating again yields **nothing**: an empty sequence, no error. Generators are **single-pass**. To go again, call the generator function again.

Proof (real output)

```
def gen():
    yield 1
    yield 2

g = gen()
print(list(g))   # [1, 2]
print(list(g))   # []      <- exhausted, silently empty
```

Why it works that way

- Calling `gen()` creates a **frame object** (the function's locals and instruction pointer) wrapped in a generator. `next()` resumes that frame. `yield` saves the frame's state and returns control. That is why local variables survive between values.
- **Nothing in the body runs until the first `next()`**. Errors inside a generator surface lazily, at iteration time, not at the call.
- **Generator expressions** `(x*x for x in data)` are the one-line form. `[...]` builds a list now; `(...)` builds a lazy generator.

- `yield from sub()` delegates to another generator and forwards values (and the return value) transparently.
- Advanced: `gen.send(value)` pushes data **into** a paused generator, `gen.throw()` injects an exception, `gen.close()` raises `GeneratorExit` at the `yield` so `finally` blocks run. This machinery preceded `async/await`.

The log-streaming example

```
import json
from itertools import islice

def read_lines(path):
    with open(path) as f:              # file stays open while the generator is alive
        for line in f:                 # file objects are themselves lazy iterators
            yield line

def parse(lines):
    for line in lines:
        try:
            yield json.loads(line)
        except json.JSONDecodeError:
            continue                    # skip bad lines instead of crashing the pipeline

def failures(events):
    for e in events:
        if e.get("level") == "ERROR":
            yield e

pipeline = failures(parse(read_lines("app.log")))
first_ten = list(islice(pipeline, 10))  # reads only as much of the file as needed
```

Each stage pulls one item from the previous stage. Memory stays flat regardless of file size, and stopping at 10 results means the rest of the file is never read. It composes like a Unix pipe.

Batching a stream (common before calling an embedding API):

```
from itertools import islice

def batched(iterable, n):
    it = iter(iterable)
    while batch := list(islice(it, n)):    # walrus needs 3.8+
        yield batch
```

(3.12 added `itertools.batched`.)

Pitfalls to name (this separates senior answers)

Pitfall	What happens	Fix
`list(gen)` on a huge stream	Defeats the point, loads everything	Iterate, don't materialise
Iterating twice	Second pass is silently empty	Re-create the generator; `itertools.tee` buffers (so it costs memory); or use a list if you truly need re-iteration

Pitfall	What happens	Fix
Resource held open	The `with open(...)` stays open until the generator is exhausted or garbage-collected	Wrap consumption in `try/finally`, call `gen.close()`, or use `contextlib.closing`
Lazy errors	Exception appears far from where you built the pipeline	Validate arguments eagerly in a normal function that then returns the generator
Two consumers on one generator	They steal each other's items	One consumer per generator, or fan out via a queue
Debugging	Can't `len()` or index it	`islice` to peek, or count items as they pass

Async generators (relevant to streaming LLMs)

Streaming tokens from a model is an **async generator**:

```
async def stream_answer(prompt):
    async for chunk in client.stream(prompt):    # each chunk arrives over the network
        yield chunk.text
```

Consumed with `async for`. Same lazy idea, but each step can `await` I/O. This is how you forward tokens to the UI as they arrive instead of waiting for the full completion.

Where it bites in your work

- Splunk results, GitHub paginated results (issues, PR comments, Actions runs) and large repo file trees are all "many items, don't hold them all". A paginator written as a generator that `yield`s each page's items is the natural shape.
- Don't quote Splunk endpoint details unless you've actually used them. Say you'd use paginated or streaming search results and iterate lazily.

Follow-ups he may ask

- "*Generator vs iterator?*" An iterator is any object with `__iter__` and `__next__`. A generator is an iterator created by `yield`, the easiest way to write one. A class-based iterator is the same thing written by hand.
- "*What does `return` do inside a generator?*" Ends it. The returned value rides on the `StopIteration` exception's `.value` and is what `yield from` evaluates to.
- "*Memory of a generator?*" Constant: one frame plus whatever you hold inside it.
- "*When is a list better?*" When you need `len`, indexing, multiple passes, or sorting.

Q5. A `@retry` decorator, `functools.wraps`, and what must not be retried

The 30-second answer

A decorator is a function that takes a function and returns a replacement. `@retry(max_attempts=3, backoff=2)` has **three layers** because `retry(...)` is a factory that returns the actual decorator, which returns the wrapper. `functools.wraps` copies the original function's identity (name, docstring, module) onto the wrapper so introspection still works. And you must **not blindly retry non-idempotent writes**: if a GitHub comment POST times out, the write may have succeeded, and retrying posts it twice.

The code (tested)

```
import functools
import random
import time

def retry(max_attempts=3, backoff=2, base_delay=0.5, exceptions=(Exception,)):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return fn(*args, **kwargs)
                except exceptions:
                    if attempt == max_attempts:
                        raise                                   # out of attempts: surface the real
error
                    cap = base_delay * backoff ** (attempt - 1)  # 0.5, 1.0, 2.0 ...
                    time.sleep(random.uniform(0, cap))           # full jitter
        return wrapper
    return decorator


@retry(max_attempts=3, exceptions=(ConnectionError,))
def flaky():
    """Does a flaky thing."""
    ...
```

I ran it with a function that fails twice then succeeds: it returned `"ok"` after 3 calls, and `flaky.__name__ == "flaky"` with the docstring intact.

Reading the three layers

```
retry(max_attempts=3)      -> returns decorator          (layer 1: takes the config)
decorator(fn)              -> returns wrapper            (layer 2: takes the function)
wrapper(*args, **kwargs)   -> runs fn with retry logic   (layer 3: runs on every call)
```

`@retry(max_attempts=3)` is just `flaky = retry(max_attempts=3)(flaky)`. Plain `@retry` with no parentheses would pass `flaky` as `max_attempts` and break. To support both forms, make the first argument optional and branch on `callable()`.

What `functools.wraps` does, and what breaks without it

`functools.wraps(fn)` applied to the wrapper copies `__module__`, `__name__`, `__qualname__`, `__doc__` and `__annotations__`, updates `__dict__`, and sets `wrapper.__wrapped__ = fn`, which `inspect.signature` follows so the signature reports the original arguments.

Without it:

- `flaky.__name__` is `'wrapper'`, the docstring is gone, and tracebacks, logs and metrics all say `wrapper`.
- **Frameworks that read the function's name, docstring or signature break.** This matters to you: LangChain's `@tool` builds the tool's name, description and argument schema from the function's name, docstring and type hints. FastAPI builds routes and OpenAPI the same way. A decorator that hides these yields a tool called `wrapper` with no description. For an agent that chooses tools by description, that is a real bug.

Async version (you need this in real agent code)

A sync wrapper around an `async def` returns the coroutine un-awaited and "succeeds" instantly, so the retry never triggers. You need a separate async wrapper:

```
import asyncio, functools, random

def aretry(max_attempts=3, backoff=2, base_delay=0.5, exceptions=(Exception,)):
    def decorator(fn):
        @functools.wraps(fn)
        async def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return await fn(*args, **kwargs)
                except exceptions:
                    if attempt == max_attempts:
                        raise
                    await asyncio.sleep(random.uniform(0, base_delay * backoff ** (attempt - 1)))
        return wrapper
    return decorator
```

Use `asyncio.sleep`, never `time.sleep`, inside async code. `asyncio.CancelledError` is not an `Exception` subclass since 3.8, so `except Exception` won't swallow it, which is correct.

Why jitter

Without jitter, 1,000 clients that failed together all retry at exactly 0.5s, then 1s, then 2s: a **thundering herd** that re-crashes the recovering service. Randomising the delay spreads them out. "Full jitter" (`uniform(0, cap)`) is the common, well-studied choice.

What should NOT be retried

Retry only **transient** failures where retrying is **safe**.

Situation	Retry?	Reason
Connection refused / DNS failure **before the request was sent**	Yes	Nothing happened on the server
HTTP 429	Yes, honouring `Retry-After`	Server told you to wait
HTTP 502 / 503 / 504	Yes for idempotent calls	Usually transient

Situation	Retry?	Reason
HTTP 400 / 401 / 403 / 404 / 422	**No**	The same bad request fails the same way; repeated 401/403 can trigger lockouts or alarms
Pydantic `ValidationError` on your own input	**No**	Deterministic bug
Timeout **after** a non-idempotent write was sent	**No, not blindly**	The write may have succeeded. The outcome is *unknown*
`KeyboardInterrupt` / `CancelledError`	**No**	Must propagate

The non-idempotent GitHub write (your "exactly-once" design)

The dangerous case is **ambiguous failure**: you POST a comment, the connection times out, and you cannot tell whether GitHub stored it. A blind retry means a duplicate comment, a second label event, or a second branch/PR. For a bot that people read, duplicates are visible and erode trust.

Ways to make a retry safe:

1. **Read-before-write (check, then act).** Embed a hidden marker in the body, for example `<!-- resolveflow:run=abc123:step=comment -->`. Before (re)posting, list the issue's comments and look for that marker. If found, the earlier attempt succeeded, so do nothing. This gives *effectively-once* behaviour on an API with no native idempotency key.
2. **Deterministic identifiers.** Derive the action's id from run + step, not a random UUID per attempt, so every attempt is recognisably the same logical action.
3. **Prefer naturally idempotent operations.** "Set label X" and "set assignee to Y" are idempotent (applying twice equals once). "Append comment" is not. Design the write layer around the idempotent forms where you can.
4. **Persist intent before acting** (outbox-style): record "about to do step N" in the checkpoint, write, then record done. On resume you know to verify rather than blindly repeat.
5. **Only the write layer retries, and only with these guards.** Reads (fetching issues, search) are safe to retry freely.

I'm not certain whether GitHub's REST API offers a generic `Idempotency-Key` header. As far as I know it does not, which is why you build the marker pattern yourself. Say exactly that: "as far as I know, so I design for it not being there."

Your repo has an exactly-once GitHub write design under `docs/exactly-once`. Re-read it before the interview so your spoken answer matches what you wrote, and be clear about which parts are implemented and which are design.

Retry amplification (senior-level point)

If layer A retries 3 times and calls layer B which retries 3 times and calls the SDK which retries 3 times, one user action can produce 27 calls against an already-struggling dependency. Pick **one** layer to own retries (usually the outermost one, which knows about idempotency), keep the inner ones at zero, add an overall **deadline/budget**, and consider a **circuit breaker** so you stop hammering a dependency that is down.

Follow-ups he may ask

- *"Make it work for both sync and async."* Detect with `inspect.iscoroutinefunction(fn)` and return the matching wrapper.
- *"What do you log?"* Each retry with attempt number, exception type, delay, and the correlation/trace id. Never log tokens or request bodies containing user data.
- *"Retry the LLM call?"* Yes for 429/5xx/timeouts (reads are safe). There is also a different kind of retry: the output failed schema validation, so you re-ask with the validation error appended, capped at 1–2 repairs, then fall back to human review rather than loop forever.
- *"How does it differ from tenacity?"* `tenacity` is the production library (stop conditions, wait strategies, retry predicates, async support). Knowing to use it, and being able to write the core idea in 10 lines, is the right answer.

One-page recall sheet

Q	One-line takeaway	The detail that proves depth
1	TypedDict = hint only, dataclass = boilerplate, BaseModel = enforced + schema	Validate at trust boundaries; LangGraph reads `Annotated` reducers off the state annotations
2	Defaults evaluated once at `def` time	`__defaults__`; becomes a cross-tenant leak; `None` sentinel, `default_factory`, `ContextVar`
3	GIL: one thread runs bytecode at a time; released on I/O	`counter += 1` still races; asyncio stalls on any sync call; I/O-bound work is unaffected
4	`yield` freezes the frame; lazy, single-pass, O(1) memory	Exhausted generator is silently empty; lazy errors; `close()` runs `finally`; async generators for token streams
5	Three layers; `wraps` preserves identity	`@tool` reads name/docstring/signature; retry only safe failures; ambiguous write = verify via marker, not blind retry

Self-test (cover the answers above)

1. Why does `@dataclass class A: x: list = []` fail, and what is it protecting you from?
2. Two threads each do `count += 1` 1M times. Why can the total be wrong *with* the GIL?
3. You call `time.sleep(2)` inside an `async def`. What happens to the 50 other tasks?
4. You loop over a generator twice and the second loop prints nothing. Why, and what are two fixes?
5. Your `@retry` is applied to an `async def`. Why does it never retry, and what does the caller get back?
6. An LLM returns JSON that fails Pydantic validation. Name two differences between "retry the call" and "repair with the error".
7. A POST to GitHub times out. Walk through exactly how you decide whether to send it again.

Next rounds (pending)

- **Round 2:** event loop internals, `gather` vs `TaskGroup`, cancellation, timeouts, backpressure with `asyncio.Queue`
- **Round 3:** context managers and `contextlib`, descriptors and `property`, `__slots__`, the data model (`__eq__` / `__hash__`), refcounting vs cyclic GC, `weakref`
- **Round 4:** design exercises (per-tenant rate limiter, LRU cache with TTL, idempotent job runner) and a walk through your own ResolveFlow code